

Обучение простой полносвязной модели копированию данных

1. Алгоритм выполнения

1. Создать сабмодуль репозитория `git@github.com:YulieRu/PINN2026-1.git` в своем репозитории. В директории `PINN2026-1/tasks/1_copy_func_model/data` выбрать файл с данными (файлы должны отличаться).

2. Импортировать модули:

```
import numpy as np
import matplotlib.pyplot as plt
import time
import torch
import torch.nn as nn
import tqdm
from scipy.interpolate import griddata
```

3. Визуализировать данные на равномерной сетке (рис. (1)).

```
np.loadtxt
np.linspace
np.meshgrid
griddata
plt.pcolor
```

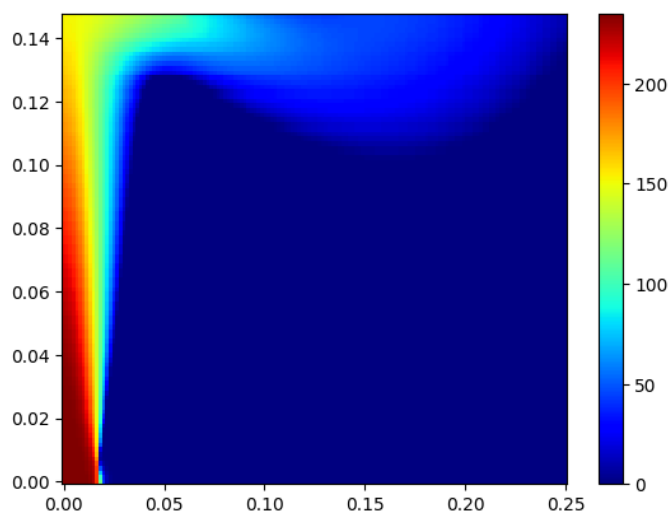


Рис. 1 – пример визуализации

4. Нормализовать данные, чтобы значения лежали в пределах от -1 до 1.

5. Создать модель, состоящую из 4 скрытых слоя по 5 нейронов, с функцией активации tanh. Пример:

```
class PINNModel(nn.Module):
    def __init__(self,
                  input_size = 1,
                  nnodes = 30,
                  output_size = 1,
                  nhlayers = 20,
                  activation=nn.Tanh(),
                  ):
        super().__init__()
        self.activation = activation
        self.nhlayers = nhlayers
        self.nnodes = nnodes

        self.dense0 = nn.Linear(input_size, nnodes)
        self.hidden_layers = nn.ModuleList([
            nn.Linear(nnodes, nnodes) for _ in range(nhlayers)
        ])

        self.output_layer = nn.Linear(nnodes, 1)

    def forward(self, x):
        if x.dtype == torch.float64:
            x = x.float()
        x = self.activation(self.dense0(x))

        for layer in self.hidden_layers:
            x = self.activation(layer(x))

        x=self.output_layer(x)
        x=self.activation(x)

        return x
```

так же можно посмотреть здесь.

6. Проверьте, что нейросеть корректно работает на тензоре из случайных чисел.
7. Подготовьте данные для обучения – приведите к корректному размеру, приведите к тензору.

```
np.flatten
torch.from_numpy
```

8. Напишите цикл для обучения . Во время цикла сохраняйте историю Loss. В конце сохраните веса модели. Пример:

```
def train(model, opt, trp: TrainParams):
    pbar = tqdm.tqdm(range(trp.numEpoch), desc='Training_Progress')
    for step in pbar:
        def closure(): # вычисление одного шага алгоритма оптимизации
            # Обнуление градиентов (по умолчанию – градиенты
            # накапливаются)
            opt.zero_grad()
            loss = copyLoss(model, trp)
```

```

#градиенты функции потерь по всем параметрам модели
loss.backward()
return loss

#Оптимизатор вызывает closure, получает градиенты и изменяет
параметры модели
current_loss = opt.step(closure).item()
hist_loss.append(current_loss)
if step % 2 == 0:
    pbar.set_description("Step: %d | Loss: %.10f" %
                        (step, current_loss))

#сохранение весов модели
torch.save(model.state_dict(), 'pinn_model')

return 0

```

9. Выведите график обучения (см. рис. (2))

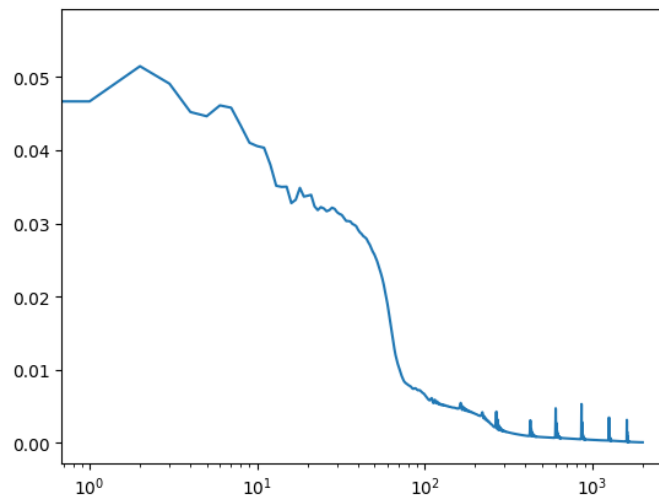


Рис. 2 – График обучения

10. Выведите исходную функцию, предсказания модели, и среднеквадратичное между ними, а также значение MSE (см. рис. (3)).

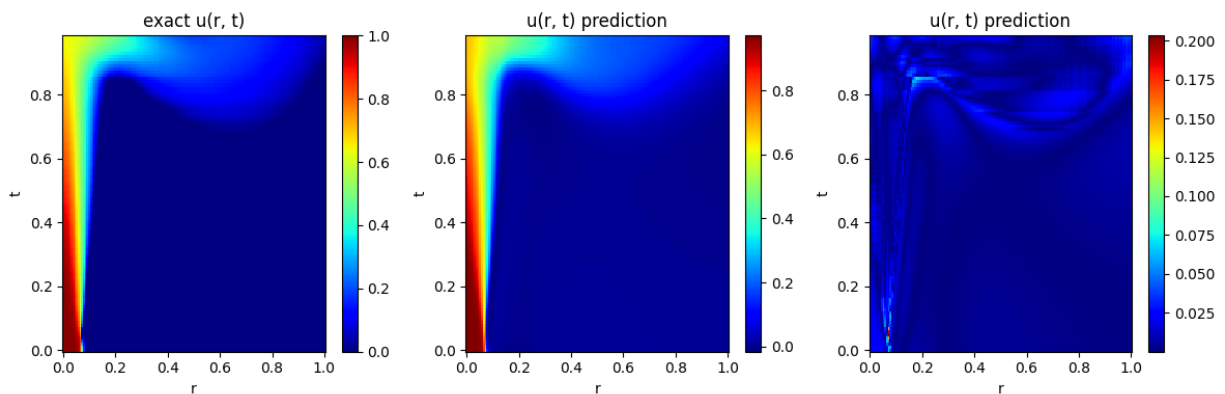


Рис. 3 – исходные данные, предсказания модели, СКЗ

2. ДЗ

Выберите любую из ниже заданных функций и обучите нейросеть копировать ее. Функции должны у всех различаться.

$$u(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (1)$$

$$u(x, y) = \exp\left(-\frac{(x-a)^2 + y^2}{2\sigma^2}\right) + \exp\left(-\frac{(x+a)^2 + y^2}{2\sigma^2}\right) \quad (2)$$

$$u(x, y) = \sin(k_x x) \cos(k_y y) \exp(-\alpha(x^2 + y^2)) \quad (3)$$

$$u(x, y) = \sin\left(k\sqrt{x^2 + y^2}\right) \quad (4)$$

$$u(x, y) = \sin\left(k\sqrt{x^2 + y^2}\right) \exp\left(-\alpha\sqrt{x^2 + y^2}\right) \quad (5)$$

$$u(x, y) = x^2 - y^2 \quad (6)$$

$$u(x, y) = \tanh\left(\beta(x^2 - y^2)\right) \quad (7)$$

$$u(x, y) = \sin(kx) \sin(ky) \quad (8)$$

$$u(x, y) = \sin(kx) \cos(ky) + \frac{1}{2} \cos(2kx) \sin(2ky) \quad (9)$$

$$u(x, y) = \sin\left(k\sqrt{x^2 + y^2} + \lambda \arctan 2(y, x)\right) \quad (10)$$

$$u(x, y) = \sin(3x) \cos(3y) + 0.5 \sin(7x + 2y) + \exp(-2(x^2 + y^2)) \quad (11)$$